

Creator guide and reference

TextPhone for Voyage | Creator guide 0.6.0

Local testing, configuration and Voyage installation are separate workflows. Follow the complete merge and readback steps before a fresh game.

Contents	Page
A phone inside the story	2
Customize and rebuild	3
Merge your world export	4
Add the result in Voyage	5
Choose the settings that matter	6
Turn any app on or off	7
Add a custom app	8
Business lists and generated menus	9
Delivery rules belong to each app	10
Hidden apps and real unlock triggers	11
Connect the host systems	12
Code and integration points	13
API and persistence reference	14
Check your installation and troubleshoot	15
Optional improvements and sources	16
Creator installation checklist	17
License and credits	18

A phone inside the story

TextPhone 0.6.0 | Creator and API guide

Give players a phone they can open in chat: installed apps, changing shortcuts, an App Store, nested business lists and narrator-generated service pages. A restaurant can lead to its menu, a job board to an offer, and a contact app to a conversation while the surrounding game continues.

Scripts own the installed-app list, shortcuts, selected app, configured directory entries, discovery filters, layout preference and reveal flags. The narrator presents those screens and generates page contents. The host game owns purchases, items, messages, appointments, contracts, quests and their consequences.

This is a single-player narrative interface. It is not a separate web app, a fixed-width phone image, a transaction engine or an authoritative alarm scheduler. The current adapter leaves phone state unchanged when it cannot identify exactly one player.

Player input	Result
phone	Open the installed-app screen
D	Open the app currently assigned D, from Home
phone store	Show available downloads
phone open APP NAME	Open an installed app
phone view PAGE	Browse a configured page or contextual detail
phone layout single / paired	Change the stored Home layout
close phone	Close the phone interface

Suggested first test: phone → phone store → phone open Maps → phone view nearby. Ordinary narration and the host clock continue; browsing can consume story time.

Customize and rebuild

Optional: skip this page when using the supplied installer unchanged.

1. Download the full generic source from github.com/Morberis/textphone-for-voyage using Code > Download ZIP. Extract it on your computer. Node.js 20 or newer is required for rebuilding.
2. Copy `config/generic-phone.json` from this creator package into the source project's config folder. Open that file in a text editor. The following pages explain app settings, examples and APIs.
3. Make your changes and save the configuration. Keep app IDs stable, use exact world location IDs, and give new apps distinct IDs and labels.
4. Open a terminal in the source folder containing `package.json`. Run the two commands below, entering each command on ONE line. No npm install or API key is needed.
5. Use the new `build/generic-phone-installer.html` to prepare a fresh complete world export. Follow the merge and installation pages again, then check your changes in a fresh game.

```
node native/build-mod.mjs config/generic-phone.json  
  build/generic-phone-mod.json
```

```
node native/build-installer.mjs build/generic-phone-mod.json  
  build/generic-phone-installer.html
```

Examples live in the source project's `examples` folder. Edit configuration and readable source, then rebuild; direct changes to a generated installer are overwritten by the next build.

Merge your world export

Adapt the included example configuration to your world before using this installer.

1. Extract the creator ZIP on your computer. Open its `textphone-for-voyage` folder. You should see the `build` folder inside it.
2. In Voyage, make a test copy using `More > Remix` on the source world, or `More > Duplicate` on your own world. Give the copy a `TEST__` name and confirm you are editing that copy.
3. In the test copy's Studio, select `Content`, then the top-level `Content` entry in the left tree. The breadcrumb should read `Content`. Select `View JSON` if the editor shows a form.
4. Select `Download initial GameState JSON` above the editor. Keep this complete export unchanged as your backup. It must be the whole world, not a running save or just the `Triggers` section.
5. On your computer, open `build/generic-phone-installer.html` from the extracted package in your browser.
6. Under `Original world JSON`, choose the complete export saved in step 4.
7. Select `Prepare updated world`. Review the result and record counts. If an error appears, stop here and resolve it; do not install a partial result.
8. Select `Download updated world JSON`. Keep this new file separate from your original backup. This is the complete world with `TextPhone` added.

Next: follow `Add the result in Voyage` on the next page, using the `NEW` file from step 8. The merge has not yet changed your live world.

The merger updates `phone_mod_` triggers and adds the narrator bridge while retaining other world fields. A different existing phone bridge requires review. Use only one phone profile per world.

Add the result in Voyage

Install the complete merged definition into the identified test copy.

1. Open with-phone.json in a text editor and copy its entire contents. For the browser route, use Copy complete JSON or open the downloaded file. Do not copy the partial build/*-phone-mod.json: it contains only phone trigger records.
2. Return to that test copy in Voyage Studio. Select Content, then the top-level Content entry so the breadcrumb is just Content. Choose View JSON if the editor is in form mode. Confirm you are in the complete initial GameState editor, not a single trigger or a running save editor.
3. Click inside the JSON text, press Ctrl+A (Cmd+A on macOS) while that editor is focused, and paste the complete prepared JSON. Use Save in the Studio toolbar. This is an editor replacement using the merged complete file, not a file upload or a Mods > Apply step.
4. Reload Studio and download the saved complete world again. Compare it with your prepared file before playtesting. With the source project, use the command below; without it, use a JSON-aware comparison tool. Review any differences before continuing.
5. Start a fresh game from the test world. Name and record the save as a test where the interface allows it. Send phone, then phone store, then a shortcut actually displayed on Home. Open each configured app, test its directory and toggles, and continue ordinary play. Check stored state as well as the narrator's description of consequences. Existing saves are not automatically updated. Use the supplied merger for this release. Purchases, payments and messages remain the responsibility of the world and narrator.

```
node native/verify-world.mjs local-worlds/with-phone.json  
    local-worlds/saved-world.json
```

Enter the comparison command on ONE line. Without Node, compare the prepared and saved complete JSON in a JSON-aware diff tool; counts alone cannot establish full equality.

06

Choose the settings that matter

Edit `config/generic-phone.json`, then rebuild. The starting apps are Messages, Maps, Food Delivery and App Store. Clock is configured but disabled; creators can enable it after connecting a suitable time system. Corner Cafe is an example: replace its catalogue entry with a business from your own world.

Setting	Meaning
<code>columns</code>	Default Home layout for a new phone: <code>false</code> =single, <code>true</code> =paired. Existing players keep their stored choice.
<code>enabled</code>	Per app: <code>false</code> omits it from the built mod; <code>true</code> or omitted enables it. Includes App Store.
<code>initialInstalledIds</code>	Enabled app IDs installed on a new phone. Include App Store when enabled.
<code>storeAppId</code>	The app that opens the scripted download catalogue.
<code>appStoreVisibility</code>	Per app: <code>public</code> , or <code>triggered</code> until revealed. This is separate from business discovery.
<code>removable</code>	Per app: <code>false</code> protects it from uninstall. App Store is always protected.
<code>directory.storeListingMode</code>	Per app: <code>known</code> , <code>all_public</code> , or <code>curated</code> .
<code>delivery.blockedLocations</code>	Per app: exact location/area exclusions and optional messages.
<code>clockStorageKey</code>	Optional name of a compact public clock record in host script storage.

Use stable IDs when renaming labels. The app label is what players type; the ID owns saved installation state. Reordering the catalogue does not intentionally reassign an existing valid shortcut. A removed configuration ID is pruned from that player's phone the next time it is processed.

Known means the world regards a named entity as known; it does not mean visited. The current native adapter rejects the visited listing mode. Device possession is not required and no invented device-availability flag is used.

Turn any app on or off

Every app, including App Store, supports `enabled:true` or `enabled:false`. Omitted means true for older configurations. Disabled apps are excluded during the build: their app instructions and business data are not shipped. A story reveal cannot enable an excluded app.

Use `enabled:false` to turn an app off. Use `enabled:true` with triggered visibility for an app that should exist but remain hidden until an event. Starting installation and player uninstall protection remain independent.

Disabling App Store removes its Home control and disables install/remove commands. Other preinstalled enabled apps still work. The Store must be in `initialInstalledIds` when enabled.

Rebuild both generated files, merge into a test world and start a fresh save. Existing world saves are not automatically updated. After an explicit script update, removed IDs are pruned at the next phone request; re-enabling does not reinstall them in that existing registry.

```
"enabled": false,
"appStoreVisibility": "public",
"removable": true
```

Intent	Settings
Off entirely	<code>enabled:false</code>
Downloadable	<code>enabled:true; visibility public; omit initial ID</code>
Start installed	<code>enabled:true; include initial ID</code>
Story unlock	<code>enabled:true; visibility triggered; omit initial ID</code>
Clock	Starts off. Enable after connecting a suitable host time system.

The switch controls this module. Independent world lore and narration may still mention the service. Do not treat it as a guaranteed narrative ban.

Add a custom app

Add an object to the apps array with a unique id, label and order. This example adds a transit app that can be downloaded. To preinstall it in a new game, also add transit to initialInstalledIds.

```
{
  "id": "transit", "label": "Transit", "order": 20,
  "preferredShortcut": "T",
  "summary": "Local routes and ticket requests.",
  "pages": [
    { "name": "Routes", "kind": "generated",
      "guidance": "Show established nearby routes and fares.",
      "contextReader": "location" },
    { "name": "Request ticket", "kind": "generated",
      "startsActivity": true,
      "guidance": "Ask for destination; quote fare; await confirmation." }
  ]
}
```

Generated pages use phone view routes. An activity page uses phone choose request ticket. Opening the app itself does not select the activity.

The short summary appears in the Store and opening menu. Give it a concrete purpose: a name alone can lead the narrator to invent the wrong kind of service. Put general service rules in app.guidance and page-specific work in page.guidance. Keep the host's existing rules in force.

Use examples/transit-app.json for copying. To continue inside a generated menu, the player can use ordinary language. A generated phone view detail also retains the current app context, but it is model-mediated browsing rather than a new deterministic transaction command.

Business lists and generated menus

A directory is a curated set of candidate businesses attached to one app. All public means all entries marked public in that configured set; it does not scan every store in the world. The same world location can participate in several apps with different policies.

```
"directory": {
  "heading": "Stores", "contentLabel": "menu",
  "currency": "credits", "storeListingMode": "known",
  "entries": [
    {"id": "corner-cafe", "label": "Corner Cafe",
     "locationId": "Corner Cafe", "public": true,
     "description": "Tea, sandwiches and pastries."},
    {"id": "harbour-cafe", "label": "Harbour Cafe",
     "locationId": "Harbour Cafe", "public": true,
     "description": "Breakfast and hot drinks by the docks."}
  ]
}
```

Mode	Visible entries
known	Public entries whose exact locationId passes known-entity
all_public	Every public entry in this app's configured catalogue
curated	Public entries also listed in directory.curatedIds

For a curated list, add "curatedIds": ["corner-cafe"]. Set public:false to exclude a candidate from every listing mode. Add a page with kind:directory and optionally use openPage to open it immediately.

The listed number selects that business. The narrator then generates offerings and indicative prices. Numbers inside that generated menu are interpreted by narration and do not select another business. Back returns from menu to list, then from list to Home.

Descriptions are catalogue data. Explain what the business sells; do not hide engine instructions in them. Generated offerings and prices are not cached by this module. Use examples/store-app.json.

Delivery rules belong to each app

Configure exclusions independently for each app. An area-specific message takes precedence over a whole-location rule; otherwise the app message is used. Location and area IDs must match the host's native values exactly.

```
"delivery": {  
  "blockedMessage": "Food Delivery cannot reach this building.",  
  "destinationRequiredMessage": "Choose a delivery address.",  
  "blockedLocations": [  
    {"locationId": "Research Tower"},  
    {"locationId": "Research Tower", "areaId": "secure-lab",  
      "message": "Use the public reception pickup point."}  
  ]  
}
```

phone delivery check tests the player's current location and area in the currently open delivery app. It reports an exclusion, a missing destination, or that the destination is not excluded. The check does not place a new order. Not excluded is narrower than available: opening hours, stock, service coverage, funds, weather, fees and arrival estimates still belong to the host service. Natural-language ordering receives the app's exclusions as context. An alternative destination must be checked under those same rules.

This module can deterministically select and emit the refusal message. It cannot guarantee that every host model will obey it or veto every native purchase writer. Treat it as an integration policy, not an atomic financial guard.

There is no shared global blacklist. Giving Food Delivery a blocked location does not block a shopping app. Use `examples/store-app.json` for a complete editable app; configure both apps separately if both should be restricted.

Hidden apps and real unlock triggers

For an enabled app, set `appStoreVisibility:triggered` and leave it out of `initialInstalledIds`. The scripted catalogue omits it and the registry rejects a premature installation request. Live narration can still invent a successful installation: this is visibility/state control, not a guaranteed prose restriction. A reveal makes it available; it does not install it or grant account access.

```
const names = Object.keys(check({type: "player-level"}) || {});
if (names.length === 1) {
  const playerKey = JSON.stringify(names[0]);
  storage.phone_mod_unlocks = storage.phone_mod_unlocks || {};
  const grants = storage.phone_mod_unlocks;
  grants[playerKey] = grants[playerKey] || {};
  grants[playerKey].signal_vault = true;
}
```

Place that script in a creator-authored trigger with the actual quest, event or other supported condition that should reveal the app. The ID `signal_vault` must equal your app ID. Use the existing player key exactly; it includes JSON string quoting.

Alternatively, set `unlockKnownEntity` to an exact host entity name. A true known-entity result reveals the app when its phone is next processed. Reveal is persistent in this release; turning a grant back to false does not revoke a reveal that was already recorded.

Public download visibility, installed state and service authorization are separate. A public job app can still require membership before accepting work. Use app/page guidance to connect that service to the world's existing access rules.

`examples/hidden-app.json` and `examples/reveal-trigger.json` provide complete records. Replace the example quest condition with your world's real trigger. The adapter currently requires exactly one player; this example is not multiplayer attribution.

12

Connect the host systems

Connection	Current implementation
Player identity	Exactly one action string and one key from check(player-level); state is keyed by JSON.stringify(player name).
Location	check(party-location) and check(party-area); used by Maps and delivery checks.
Discovery	check(known-entity) for each configured exact locationId. No native historical-visit reader.
Clock	Disabled by default. Scripts cannot read the built-in clock. Optional clockStorageKey reads a creator-provided custom storage record for Time page context.
Orders and money	Host-generated transaction and state systems; the phone does not charge, pay or maintain a second ledger.
Messages and contracts	Host story/state systems; existing relationships, access rules and explicit choices remain relevant.
Living Continuity	May supply time and continuity through the host. The generic phone has no required LC dependency.

To enable Clock, find app id clock in your profile config and set enabled:true. Its supplied public visibility then allows downloading. Add clock to initialInstalledIds only if it should start installed in new games. Rebuild the mod and installer. LC installation alone does not configure the connection.

For an optional clock adapter, write a small public record such as storage.public_clock = {date:"2040-05-03", time:"09:20"}, then set clockStorageKey:"public_clock". Keep that record short: at most 1000 serialized characters are included. The host adapter must update it before the relevant phone turn; this module does not infer a civil date from game-tick.

Clock Alarms is a generated view, not a new scheduler. If your world requires dependable alarms, connect an existing authoritative event system and verify both the stored event and its later firing. An attractive generated alarm list is insufficient evidence.

To advance to a specific time, enter Advance time to [time] in the normal story input, replacing [time] with your target time. To wait a duration, enter Wait [number] hours, replacing [number] with the hours to pass. The Clock Time page displays these instructions. The host handles time passage and intervening events.

There is no inventory-based device gate. Renaming a player can create a different phone key. Multiplayer is unsupported.

Code and integration points

Source / public interface	Responsibility
native/build-mod.mjs : buildPhoneMod(config)	Validate configuration and build inert data records plus six native handlers. Returns {triggers}.
src/merge-phone-world.mjs : mergePhoneWorld(world, mod, bridge)	Return a separate complete world and preservation report; reject saves and conflicting bridges.
mod-context.mjs	Read configuration, attribute the player, restore/prune phone state, allocate stable shortcuts and build screen context.
mod-home / mod-marketplace	Home, layout, close/back, install, uninstall and hidden download visibility.
mod-services / mod-directory	Configured pages, activity choice, directory selection and generated-menu context.
mod-delivery / mod-follow-through	Per-app policy checks and context for natural or generated detail requests.

The builder emits `phone_mod_config_*` and `phone_mod_data_*` records as JSON inside inert script comments, read with `JSON.parse`. Runtime triggers use action-text conditions, planning phase and party scope. They use the documented storage, triggers, check, effects and log surfaces; they do not evaluate generated code. Only edit readable source/configuration, then rebuild. Deployment scripts remove indentation/comments to fit native limits; complete world exports stay pretty-printed. In Studio the generated records are under Content → Mechanics → Advanced → Triggers.

These source files are in the generic GitHub project. The creator package contains your editable profile and the already-built installer.

API and persistence reference

API or field	Contract
phone install / uninstall APP NAME	Case-insensitive exact app label; updates installation revision. Existing host orders/accounts are not deleted by script.
phone view PAGE	Browse a configured page. An unregistered detail continues inside the current app as model-mediated browsing.
phone choose PAGE	Invoke a configured startsActivity:true page; ordinary page viewing does not invoke it.
phone shops / phone business NAME	Open the current app's eligible business list / select an exact listed label. phone restaurant NAME is an alias.
Back / phone back	Scripted directory menu→list→Home; from other app pages returns Home. Generated labels may claim a different destination.
phone_mod.players[playerKey]	Installed/revealed IDs, shortcuts, layout, app/page/directory, revision and lastHandled receipt.
phone_mod_unlocks[playerKey] [appld]	Creator grant:true reveals the download on the next processed phone action.

The receipt combines game tick and action text to prevent duplicate scripted navigation effects across handlers. A generated detail adds context without inventing a deterministic transaction. Natural conversation does not silently navigate the phone.

Configuration bounds: 32 apps, 12 pages per app, 128 businesses per app, unique lowercase ASCII app IDs up to 48 characters, unique short app labels up to 32 characters, and one-line summaries up to 160 characters. Page names use letters, spaces and hyphens, up to 40 characters. App/page guidance is bounded to 1800 characters each. Commands are limited to 400 characters. The builder also rejects any serialized trigger above 10000 characters.

Changing the starting app list or default layout affects new phone state. Existing state retains its installed IDs and chosen layout, apart from IDs removed from the current catalogue. Start a fresh game when changing profiles; legacy save migration is not provided.

Do not install two phone profiles together under the same namespace. Use the complete-profile merger to remove obsolete phone records. Keep unrelated trigger names outside phone_mod_.

Check your installation and troubleshoot

- 1. Start a fresh game. Enter phone, open an app using its displayed shortcut, then try phone store. Close and reopen the save to check that installed apps and preferences persist.
- 2. Check that the world's original NPCs, locations, quests and lore are still present. After configuration changes, check both sides of the options you changed, such as allowed/blocked delivery or before/after a hidden-app reveal.
- 3. For purchases and other important actions, check the game's actual inventory, currency or quest state. A narrated success is not always a recorded result.

Known boundary	Practical response
Narrator omits or changes a screen	Try an exact app command and inspect state. Home can repeat a prior invented app, so returning Home is not a guaranteed recovery.
Generated controls outside the API	Use ordinary-language follow-up. phone view details has contextual routing; other invented commands are not guaranteed.
Lost labels or changing prices	Use clear word-bearing rows and explicitly confirm the quoted total. Inspect native consequences.
Invented alarms, accounts or profiles	Connect authoritative host records if required; generated prose alone is not a stored fact.
Multiplayer	Do not advertise support with this adapter.
Native mod application	Use the verified complete-export merge route until Mods → Apply is separately checked.

Use a test copy and keep the original export. Existing saves are not automatically upgraded; multiplayer is unsupported. Normal narration continues and browsing may advance story time.

Optional improvements and sources

Creators can extend the phone without adding a new engine for every app. Start with a short purpose and one useful page, then add only the host connections the experience requires.

- Author a merchant catalogue per service, including location IDs, useful descriptions and discovery policy.
- Add a transit, journal, club, guild, weather, medical appointment or property app using generated pages and existing host systems.
- Reveal an app through a quest, invitation or known place; keep download visibility separate from account permission.
- Add an authoritative menu/cart/order or alarm adapter if exact persistence matters. This is extra implementation, not an undocumented built-in feature.
- Provide explicit profile candidates or service eligibility data where broad narrator guidance is too variable.
- Choose the starting apps and delivery messages that suit each world. A smaller catalogue usually makes the first phone interaction easier to read.

[Voyage Studio](#)

[Latitude Team Trigger Scripts Guide](#)

[Guide to Creator-Authored Quests](#)

[Voyage creation community](#)

TextPhone is independently authored and is not an official Latitude feature. MIT covers its original code and documentation; third-party world material remains separate.

Creator installation checklist

Check these steps on your own test remix. Checkmarks record your review, not a guarantee of model behavior.

- ☐ Preserve the original export; create and record a clearly named test remix.
- ☐ Use the supplied profile, or customize it for your world before rebuilding.
- ☐ Use exact native location/area IDs and distinct app names and IDs.
- ☐ Set starting apps, discovery policy and each app's delivery messages.
- ☐ Rebuild both the mod JSON and HTML installer after configuration changes.
- ☐ Paste the complete merged JSON at Content root; Save, reload and verify-world.
- ☐ Confirm original lore, NPCs, locations, quests and host triggers remain.
- ☐ Start a fresh save; open every app/page and test each chosen toggle.
- ☐ Test reveal before/after, install/remove, shortcuts and save/reload.
- ☐ Check narration AND saved state for consequential actions.
- ☐ Review the troubleshooting and gameplay limitations before sharing.

MIT for our work

MIT covers the original TextPhone code and documentation. Third-party world material and assets remain separate and are not relicensed by this project.

Not an official Latitude product. Voyage use remains subject to Latitude's applicable platform terms. The MIT license itself is unmodified.

MIT License

Copyright (c) 2026 Morberis

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

[Latitude Terms of Service](#)